Лекція №17 (2 години). Хуки ефектів: `useEffect` та життєвий цикл.

План лекції

1. Поняття "Чиста функція" (Pure Function) та "Побічний ефект" (Side Effect).
2. Життєвий цикл компонента: Монтування, Оновлення, Демонтування.
3. Хук `useEffect`: призначення та базовий синтаксис.
4. Масив залежностей (Dependency Array) та його три конфігурації.
5. Запит даних з сервера (Data Fetching) всередині `useEffect`.
6. Функція очищення (Cleanup Function) та витоки пам'яті.
7. Типові помилки: безкінечні цикли рендерингу (Infinite Loops).

Перелік умовних скорочень

- **UI** (User Interface) — користувацький інтерфейс.
- **Hook** — хук, функція для використання можливостей React.
- **REST API** — архітектурний стиль для мережевих запитів.
- **DOM** (Document Object Model) — об'єктна модель документа.

Вступ

З попередньої лекції ми дізналися, що React-компонент в ідеалі працює за формулою: *UI = f(State, Props)*. Дали йому дані — він намалював кнопки. Але що робити, якщо ці дані треба спочатку завантажити з сервера? Що робити, якщо ми хочемо запустити таймер зворотного відліку? Або якщо потрібно вручну звернутися до `document.title` і змінити назву вкладки в браузері?

Всі ці дії виходять за межі "просто намалювати UI". В термінології програмування вони називаються **Побічними ефектами (Side Effects)**.

Якщо ми спробуємо виконати запит до сервера безпосередньо в тілі компонента (класичній функції), ми випадково зламаємо весь додаток, адже функція буде запускатися знову і знову при кожній найменшій зміні. Щоб виконувати побічні ефекти безпечно, синхронно з етапами життя компонента, розробники використовують наймогутніший (і найскладніший для новачків) хук — `useEffect`.

Мета цієї лекції — зрозуміти філософію "ефектів", фази "Життєвого циклу" та навчитися робити API-запити без ризику покласти сервер безкінечними циклами.

1. Чисті функції та Побічні ефекти

React бере своє коріння з функціонального програмування. Одна з головних вимог React — **кожен компонент повинен бути Чистою Функцією при рендерингу**.

Чиста функція (Pure Function) — це функція, яка:

1. Завжди повертає однаковий результат при однакових вхідних аргументах.
2. Не змінює нічого поза своєю областю видимості (не мутує глобальні змінні).
3. Не виконує дій, результати яких непередбачувані (запит в Інтернет, `Math.random()`, читання файлу).

Все, що порушує ці правила, називається **Побічним ефектом (Side Effect)**. Але без побічних ефектів веб-додаток був би просто мертвою картинкою. Нам ПОТРІБНІ запити в інтернет та таймери. Тому React каже: "*Виконуйте ваші побічні ефекти, але робіть це у спеціальному місці*". Цим місцем є `useEffect`.

2. Життєвий цикл компонента

Як і людина, кожен React-компонент у браузері проходить через три головні фази життя:

1. **Mounting (Монтування/Народження):** Компонент вперше з'являється на екрані (додається в реальний DOM браузера).
2. **Updating (Оновлення/Життя):** Користувач взаємодіє зі сторінкою, змінюються `Props` або `State`. Компонент перераховується і перемальовує свою частину екрана. Ця фаза може повторюватися тисячі разів.
3. **Unmounting (Демонтування/Смерть):** Користувач переходить на іншу сторінку, або компонент приховується умовою (`if`), і він назавжди видаляється з DOM.

Давним-давно (у Класових компонентах) для кожної фази були окремі функції (`componentDidMount` , `componentDidUpdate`). Зараз `useEffect` дозволяє контролювати **усі три фази** одночасно.

3. Хук `useEffect`: базовий синтаксис

`useEffect` викликається на верхньому рівні компонента. Він приймає два аргументи:

1. **Callback-функцію** (сам ефект, код, який треба виконати).

2. **Масив залежностей** (необов'язковий, але в 99% випадків критично необхідний масив змінних).

```
import { useEffect, useState } from "react";

function PageTitle() {
  const [name, setName] = useState("Іван");

  // Ефект: змінюємо заголовок вкладки браузера
  useEffect(() => {
    document.title = `Сторінка користувача ${name}`;
  });

  return <input value={name} onChange={(e) => setName(e.target.value)} />;
}
```

Якщо ми **НЕ передамо** другий аргумент (як у прикладі вище), цей ефект буде спрацьовувати спочатку при Монтуванні, а потім — **ПІСЛЯ КОЖНОГО ОНОВЛЕННЯ** компонента. (Кожна надрукована буква в `input` буде викликати цей ефект).

4. Масив залежностей (Dependency Array)

Масив залежностей — це масив `[]`, який ми передаємо другим аргументом. Це пульт керування ефектом. Ви ніби кажете React-у: "*Будь ласка, запускай цю функцію знову, ТІЛЬКИ ЯКЩО змінилася хоча б одна з цих змінних*".

Існує рівно 3 конфігурації масиву залежностей:

1. Ефект без масиву взагалі (Використовується дуже рідко)

```
useEffect(() => {
  // Виконається після ПЕРШОГО рендеру і після КОЖНОГО подальшого рендеру
});
```

2. Порожній масив `[]` (Використовується найчастіше для API)

```
useEffect(() => {
  // Виконається ТІЛЬКИ ОДИН РАЗ (при Монтуванні).
  // Ідеально підходить для початкового завантаження даних (fetch).
}, []);
```

3. Масив зі змінними `[var1, var2]` (Реактивний ефект)

```
useEffect(() => {
  // Виконається при Монтуванні, а також щоразу, коли зміниться var1 або var2.
  // Якщо і var1 і var2 залишилися незмінними - ефект ігнорується.
  searchServer(query);
}, [query]);
```

> ***Золоте правило лінтера:*** *Якщо ви використовуєте всередині {тіла ефекту} будь-яку змінну (prop або state), ви* ***зобов'язані*** *вказати її у масиві залежностей. Якщо ви цього не зробите, ефект зафіксує старе, застаріле значення цієї змінної.*

5. Запит даних (Data Fetching) всередині `useEffect`

Найпопулярніше застосування `useEffect` — це зробити запит за інформацією до сервера (API) одразу після того, як сторінка завантажилася.

Оскільки `useEffect` не може бути напряму асинхронним (не можна написати `useEffect(async () => {})`), ми створюємо `async`-функцію всередині нього і одразу її викликаємо:

```
function UserList() {
  // 1. Стан для збереження фінальних даних (порожній масив)
  const [users, setUsers] = useState([]);
  // 2. Стан для індикатора завантаження
  const [isLoading, setIsLoading] = useState(true);

  // 3. Сам ефект, який запуститься 1 раз при старті
  useEffect(() => {
    const fetchUsers = async () => {
      try {
        const response = await fetch("https://api.test/users");
        const data = await response.json();

        // Зберігаємо справжні дані в State
        setUsers(data);
      } catch (error) {
        console.error("Помилка!", error);
      } finally {
        // В будь-якому випадку вимикаємо "крутилку"
        setIsLoading(false);
```

```
      }
    };

    fetchUsers();
  }, []); // ПОРОЖНІЙ МАСИВ! Це гарантія, що запит не піде по колу

  // Рендеринг UI
  if (isLoading) return <h2>Завантаження даних...</h2>;

  return (
    <ul>
      {users.map((user) => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

6. Функція очищення (Cleanup Function) та витоки пам'яті

Іноді наш Ефект залишає після себе "сміття" у системі.

Наприклад, при Монтуванні компонента-віджета ми запустили вічний таймер (через `setInterval(..., 1000)`). Коли користувач піде на іншу сторінку, компонент зникне (Демонтування). **АЛЕ таймер продовжить працювати (цокати кожну секунду)** в пам'яті комп'ютера! Це називається "Витік пам'яті" (Memory Leak). Якщо користувач зайде і вийде зі сторінки 100 разів, запуститься 100 паралельних таймерів і браузер "впаде".

Щоб прибрати за собою, `useEffect` може `return` (повернути) спеціальну функцію **Очищення (Cleanup)**. React автоматично викличе цю функцію якраз перед Демонтуванням компонента, (а також перед кожним наступним перезапуском цього ж ефекту).

```
useEffect(() => {
  console.log("Ефект: Ставимо таймер!");

  // Запускаємо щось, що працює постійно
  const timerId = setInterval(() => {
    setSeconds((s) => s + 1);
  }, 1000);

  // Функція очищення (Прибирання)
```

```
  return () => {
    console.log("Очищення: Зупиняємо старий таймер!");
    clearInterval(timerId); // Вбиваємо попередній процес
  };
}, []);
```

Ця техніка також використовується для відписки від WebSockets або скасування мережевих запитів (`AbortController`), якщо користувач натиснув кнопку "Назад", не дочекавшись завантаження файлу.

7. Безкінечні цикли (Infinite Loops)

Найжахливіший баг новачка — влаштувати DDoS атаку на сервер власноруч. Це стається, коли ви змішуєте `useEffect` і `useState` , забувши про масив залежностей.

```
// ЯК НЕ ТРЕБА РОБИТИ (код-самовбивця):
function BadComponent() {
  const [data, setData] = useState([]);

  useEffect(() => {
    fetch("/api/data").then((res) => setData(res.data));
  }); // ❌ Забули поставити [] !!!

  return <div>...</div>;
}
```

Чому це ламається (цикл смерті):

1. Компонент завантажується вперше (Монтування).
2. Запускається `useEffect` (без масиву).
3. Він робить `fetch` і в кінці викликає `setData(...)` .
4. Визклик `setData(...)` змінює State.
5. React бачить нові дані і **ПЕРЕМАЛЬОВУЄ** весь компонент (Update).
6. Під час перемальовки React знову бачить хук `useEffect` (бо масиву немає, значить треба пускати після кожного оновлення).
7. `useEffect` знову робить `fetch` і викликає `setData` .
8. State змінюється. Перемальовка! Запуск `useEffect` ! `fetch` ! `setData` ! ... І так 100 000 разів на секунду, поки браузер не зависне або сервер не заблокує вашу IP-адресу.

Завжди використовуйте масив `[]` для API запитів!

Висновки

1. Всередині (тіла) React-компонента можна виконувати лише чистий, синхронний код для прорахунку UI. Для всіх інших "небезпечних" операцій (Побічних ефектів) створено хук `useEffect`.
2. Кожен компонент проходить 3 фази: Монтування (додавання у DOM), Оновлення (зміна props/state) та Демонтування (видалення з DOM).
3. `useEffect` приймає два аргументи: функцію (що робити) та Масив Залежностей (коли це робити).
4. Якщо масив залежностей порожній `[]`, ефект запуститься лише один раз (при Монтуванні). Це стандартна поведінка для завантаження даних `fetch`.
5. `useEffect` не підтримує нативне `async` перед своєю основною callback-функцією. Асинхронні запити потрібно вкладати як внутрішні функції.
6. Для відміни довготривалих операцій (таймери, підписки, слухачі) `useEffect` повинен повертати іншу функцію (Cleanup), яка запуститься перед Демонтуванням.
7. Відсутність масиву залежностей в ефекті зі зміною State гарантовано призведе до катастрофічного нескінченного циклу перемальовок.

Джерела

1. React Docs: Synchronizing with Effects — Офіційна філософія ефектів.
2. React Docs: Lifecycle of Reactive Effects
3. Dan Abramov: A Complete Guide to useEffect — Найвідоміша (але складна) стаття від творця Redux про глибинні механіки ефектів.

Запитання для самоперевірки

1. Згадайте визначення "Чистої функції". Чому запит на сервер (`fetch`) або запуск таймера `setInterval` вважаються Побічними Ефектами, що порушують чистоту?
2. Які три основні фази має життєвий цикл кожного окремого компонента в інтерфейсі?
3. Що буде, якщо ми взагалі не передамо другий аргумент (масив) до функції `useEffect`? Коли саме він запуститься?
4. Для чого в 90% випадків ідеально підходить хук `useEffect` із порожнім масивом залежностей `[]`? Як часто він виконується?
5. Чому React видаватиме попередження в консолі, якщо ви спробуєте написати `useEffect(async () => {...}, [])`? Як правильно робити асинхронні виклики всередині цього хука?

6. Наведіть сценарій, коли функція "Очищення" (та, яку ефект повертає через `return () => {}`) є абсолютно необхідною для запобігання витоку пам'яті? Коли саме React її викличе?
7. Поясніть покроково, як відсутність порожнього масиву `[]` при виклику `fetch` (який закінчується командою `setData`) призводить до безкінечного циклу зависання браузера?